

## # Análisis de arquitectura de capas — pdf-extratext

**\*\*Fecha:\*\*** 17/08/2026

**\*\*Proyecto:\*\*** pdf-extratext (FastAPI + MongoDB + Ollama)

### ## Resumen ejecutivo

| Arquitectura | ¿Se utiliza? | Cumplimiento |

|-----|-----|-----|

| 3 capas (Layered) | Sí | Parcial — con violaciones |

| Hexagonal (Puertos y Adaptadores) | No | No hay dominio, puertos ni adaptadores |

| Onion | No | Sin capa dominio ni inversión de dependencias |

| Clean | No | Sin entidades ni casos de uso |

El proyecto implementa una **\*\*arquitectura en 3 capas\*\*** (presentación, lógica de negocio, acceso a datos) reflejada en su estructura de carpetas ( `app/` , `service/` , `repository/` , `config/` ) y en los README de cada capa. Sin embargo, existen violaciones de la disciplina de capas: la presentación accede directamente a la capa de datos y la orquestación del caso de uso principal vive en el controller.

### ## Estructura real vs. estructura documentada

#### ### Estructura real (código)

```\n

main.py            → Bootstrap de la app (FastAPI)

app/routers.py     → Endpoints HTTP (presentación)

app/models/\_\_init\_\_.py → VACÍO (sin modelos)

service/pdf\_service.py → checksum, extracción de texto, resumen IA

repository/db\_repo.py → CRUD contra MongoDB (funciones sueltas)

config/settings.py → Configuración (pydantic-settings)

config/database.py → Conexión Mongo (cliente, db, collection)

docs/clases.puml → Diagrama de clases (idealizado)

docs/secuencia.puml → Diagrama de secuencia (idealizado)

tests/test\_main.py → Tests de integración HTTP

```\n

### ### Estructura documentada (READMEs + puml)

- `app/README.md` : "Aquí entran todas las peticiones HTTP y **\*\*se delegan a la capa de servicios\*\***."

- `service/README.md` : "Procesa las peticiones recibidas desde los endpoints y **\*\*se comunica con el repositorio\*\***."

- `repository/README.md` : "**\*\*Ninguna otra capa debe acceder a la base de datos directamente.\*\***"

- `clases.puml` : declara clases `DocumentController`, `Validador`,  
`DocumentService`, `Repository`, `DocumentModel <<Entity>>` y `Ollama\_IA`  
<<External Service>>`.

La documentación describe un diseño en capas más estricto de lo que el código implementa (ver "Violaciones").

### ## Diagrama de dependencias actual

```mermaid

graph TD

subgraph Presentación

R[app/routers.py]

```

end

subgraph Lógica de negocio
    S[service/pdf_service.py]
end

subgraph Acceso a datos
    Repo[repository/db_repo.py]
end

subgraph Config / Infra
    C[config/database.py]
    St[config/settings.py]
end

subgraph Externo
    OLL[Ollama - llama3.2]
    MONGO[(MongoDB)]
end

```

```
R -->|"import directo (VIOLACIÓN)"| Repo
```

```
R --> S
```

```
S --> OLL
```

```
Repo --> C
```

```
Repo --> MONGO
```

```
C --> St
```

```

, , ,

```

La flecha `R --> Repo` es la violación principal: `app/routers.py:4-7` importa funciones de `repository.db\_repo` y las usa directamente (p.ej. `obtener\_por\_checksum` en la línea 51 y `guardar\_documento` en la línea 70), en lugar de delegar al service.

## ## Evaluación por arquitectura

### ### 3 capas (Layered)

**\*\*¿Se utiliza? Sí, de forma parcial.\*\***

Correcto:

- Separación física en presentación / negocio / datos ( `app` , `service` , `repository` ).
- La dirección general de dependencias es descendente (service → repository → config).
- `main.py` bootstrap separado de la lógica.

Violaciones:

- **\*\*Presentación → datos directo\*\***: el router importa y llama al repositorio (ver diagrama). Rompe la regla de que la presentación solo conoce a la capa de negocio.
- **\*\*Caso de uso en el controller\*\***: `POST /upload` ( `app/routers.py:33-78` ) orquesta validación, tamaño, checksum, dedupe, extracción, IA y persistencia. El service no se comunica con el repositorio como documenta su README; lo hace el controller.
- **\*\*Service de "helpers", no capa de negocio\*\***: `pdf\_service.py` expone funciones granulares ( `calcular\_checksum` , `extraer\_texto\_real` , `generar\_resumen\_ia` ) sin orquestar ningún flujo de negocio.
- **\*\*Modelos ausentes\*\***: `app/models/` está vacío; los documentos se manipulan como `dict` crudos.

### ### Hexagonal (Puertos y Adaptadores)

**\*\*¿Se utiliza? No.\*\***

- No hay **\*\*dominio\*\*** central: ``app/models/__init__.py`` está vacío, no existe entidad ``Document``.
- No hay **\*\*puertos\*\*** (interfaces): ni el repositorio ni el servicio de IA definen una abstracción que el dominio pueda consumir.
- No hay **\*\*adaptadores\*\*** que implementen puertos: ``db_repo.py`` es un adaptador concreto de Mongo sin contrato previo; ``cliente_ia = Client(...)`` vive instanciado a nivel de módulo en ``pdf_service.py:8``.
- El controller depende de clases concretas (``Client`` de ollama, funciones de mongo), sin inversión de dependencias.

### ### Onion

#### **\*\*¿Se utiliza? No.\*\***

- Falta el **\*\*núcleo de dominio\*\*** (capas más internas) y la capa de **\*\*aplicación/casos de uso\*\***.
- No hay **\*\*inversión de dependencias\*\***: las dependencias apuntan hacia infraestructura (mongo, ollama), no hacia el dominio.
- ``service/`` se comporta como infraestructura/adaptador (instancia un cliente HTTP y usa pdfplumber), no como capa de aplicación.

### ### Clean Architecture

#### **\*\*¿Se utiliza? No.\*\***

- No hay **\*\*entidades\*\*** (``models/`` vacío) ni **\*\*casos de uso\*\*** (la orquestación está en el controller).
- El flujo de ``upload`` mezcla framework (FastAPI), validación, servicio y persistencia en un solo handler.
- Todo el código está acoplado al framework y a librerías concretas (FastAPI, pymongo, pdfplumber, ollama), sin interfaces de adaptadores.

## ## Violaciones de la arquitectura en capas

| # | Violación | Ubicación | Impacto |

|---|-----|-----|-----|

| 1 | Presentación importa la capa de datos | `app/routers.py:4-7` | Rompe el flujo estricto de capas |

| 2 | Orquestación del caso de uso en el controller | `app/routers.py:33-78` | Controller "gordo", service "delgado" |

| 3 | Capa de modelos vacía | `app/models/\_\_init\_\_.py` | Documentos como dict crudos, sin contrato |

| 4 | Service con infraestructura acoplada | `service/pdf\_service.py:7-8` | Cliente IA y pdfplumber dentro de "lógica de negocio" |

| 5 | Docs no reflejan el código | `docs/\*.puml` | Declaran `Validador`, `DocumentModel`, clase `Repository` inexistentes |

## ## Recomendaciones

### ### Para consolidar la arquitectura en 3 capas

1. **\*\*Sacar la orquestación del router\*\***: crear `procesar\_archivo(contenido, filename, content\_type) -> Documento` en `service/` y que el endpoint solo delegue.
2. **\*\*Eliminar el import router → repository\*\***: el router debe hablar únicamente con el service.
3. **\*\*Modelar la entidad\*\***: crear `Document` (Pydantic/dataclass) en `app/models/` y usarlo en service y repository.

### ### Para evolucionar hacia Hexagonal/Clean

4. **\*\*Definir puertos\*\***: interfaz `DocumentRepository` y `AIService`, con `MongoDocumentRepository` y `OllamaAIService` como adaptadores.

- 5. **\*\*Inyectar dependencias\*\***: pasar repositorio y servicio de IA por constructor/handler, no instanciar a nivel de módulo.
- 6. **\*\*Extraer un `Validador`\*\*** (formato y tamaño), tal como documenta el propio `clases.puml`.

### Alineación de documentación

7. Actualizar `docs/clases.puml` y `docs/secuencia.puml` para reflejar la estructura real, o implementar las clases que documentan.

## Tabla resumen

| Aspecto   Estado                                  |
|---------------------------------------------------|
| ----- -----                                       |
| Arquitectura detectada   3 capas (Layered)        |
| Separación app / service / repository   ✓         |
| Presentación dependiendo de datos   X (violación) |
| Orquestación en service   X (en controller)       |
| Modelo de dominio   X (vacío)                     |
| Puertos / adaptadores   X                         |
| Inversión de dependencias   X                     |
| Documentación alineada   X                        |

# Análisis de principios de programación — pdf-extratext

**\*\*Fecha:\*\*** 17/08/2026

**\*\*Proyecto:\*\*** pdf-extratext (FastAPI + MongoDB + Ollama)

## Resumen general

| Principio | Cumplimiento | Nota |

|-----|-----|-----|

| DRY | No cumple | 3 violaciones claras + menores |

| KISS | Cumple parcialmente | 3 puntos de complejidad innecesaria |

| YAGNI | No cumple | 4 elementos que sobran |

## ## DRY — Violaciones

### 1. **\*\*Conversión `ObjectId` → `str` repetida 3 veces\*\***

(`repository/db\_repo.py:7`, `app/routers.py:30`, `app/routers.py:73`). Debería centralizarse en un helper de serialización.

2. **\*\*Acceso a la colección duplicado\*\*** — `config/database.py:6` define `collection = db["documents"]` pero nunca se usa; cada función de `db\_repo.py` repite `db["documents"].find/insert/update/delete`.

3. **\*\*`OLLAMA\_URL` con dos fuentes de verdad\*\*** — `config/settings.py:9` (`settings.ollama\_url`) y `service/pdf\_service.py:7` (`os.getenv`).

### **\*\*Menores:\*\***

- Patrón `try/except InvalidId` → `return None` repetido en 3 funciones del repo (`obtener\_por\_id`, `actualizar\_nombre`, `eliminar\_documento`).

- Fixture `b"Texto simulado para el test de subida"` duplicado en `tests/test\_main.py` (2 tests).

## ## KISS — Complejidad innecesaria

1. **\*\*Fallback especulativo en `obtener\_por\_id`\*\*** (`db\_repo.py:9-14`): si el ID no es un `ObjectId` válido, busca por string (`{"\_id": doc\_id}`). Camino que probablemente nunca se ejecuta; más simple sería devolver `None` sobre `InvalidId`.

2. **\*\*Config redundante e inconsistente\*\*** — ``settings.app_name`` declarado sin usar, título hardcodeado ``"PDF Extractor API"`` en ``main.py:15`` y ``APP_NAME="Extraccion PDF API"`` en el `` .env ``. Tres valores que no se conectan.
3. **\*\*`settings.ollama\_url` existe pero el service usa `os.getenv`\*\*** — duplica la configuración (mismo punto que DRY #3).

## **## YAGNI — Código que sobra**

1. **\*\*`app/models/\_\_init\_\_.py` vacío\*\*** — paquete placeholder; además ``NombreUpdate`` (único modelo real) vive en ``app/routers.py:14`` en vez de ``app/models/``.
2. **\*\*`collection` sin usar\*\*** en ``config/database.py:6``.
3. **\*\*Dependencia `pypdf2`\*\*** en ``pyproject.toml:14`` — nunca se importa (se usa ``pdfplumber``).
4. **\*\*Fallback de búsqueda por string\*\*** en ``obtener_por_id`` — funcionalidad especulativa.

## **## Recomendaciones**

1. En ``db_repo.py``: usar ``collection``, agregar ``_serializar(doc)`` y simplificar ``obtener_por_id``.
2. En ``routers.py``: usar el serializador del repo (eliminar ``doc["_id"] = str(...)`` de la línea 30).
3. En ``pdf_service.py``: usar ``settings.ollama_url``, eliminar ``os.getenv`` e import ``os``.
4. En ``main.py``: usar ``settings.app_name`` para el título de ``FastAPI``.
5. Eliminar ``pypdf2`` de ``pyproject.toml`` y ``collection`` de ``database.py``.
6. Mover ``NombreUpdate`` a ``app/models/`` y extraer el fixture de PDF duplicado en los tests.

## **## Resumen por archivo**

| Archivo | Principio | Detalle | Severidad |

|-----|-----|-----|-----|

| `repository/db\_repo.py` | DRY | `db["documents"]` repetido, `\_id`→str  
duplicado, try/except ×3 | Alta |

| `repository/db\_repo.py` | KISS/YAGNI | Fallback de búsqueda por string | Media |

| `app/routers.py` | DRY | `\_id`→str duplicado | Media |

| `config/database.py` | YAGNI | `collection` sin usar | Baja |

| `config/settings.py` | KISS | `app\_name` sin usar | Baja |

| `service/pdf\_service.py` | DRY | `os.getenv` duplicando `settings` | Media |

| `main.py` | KISS | Título hardcodeado | Baja |

| `pyproject.toml` | YAGNI | `pypdf2` sin usar | Baja |

| `app/models/\_\_init\_\_.py` | YAGNI | Paquete vacío | Baja |

| `tests/test\_main.py` | DRY | Fixture duplicado | Baja |

## # Análisis de cumplimiento de TDD — pdf-extratext

**\*\*Fecha:\*\*** 17/08/2026

**\*\*Proyecto:\*\*** pdf-extratext (FastAPI + MongoDB + Ollama)

### ## Veredicto

**\*\*No cumple TDD.\*\*** Existe una suite de tests, pero fue escrita como integración post-implementación, no como ciclo red→green. Además, los tests están rotos (no pasan contra el código actual).

### ## Evidencia de incumplimiento

#### ### 1. Los tests están rotos (no están en verde)

- `tests/test_main.py:29`` espera `"PDF subido, validado y guardado correctamente."` pero `app/routers.py:77`` devuelve `"PDF subido, validado y resumido con IA exitosamente."` → assertion desactualizada, el test falla.
- El "PDF falso" `b"Texto simulado..."`` no es un PDF válido → `pdfplumber.open`` lanza excepción en `extraer_texto_real`` → `/upload`` responde 500 → `test_subida_exitosa`` y `test_subida_duplicada`` fallan.

### ### 2. Violación de F.I.R.S.T.

- **Fast X** — conectan a MongoDB real + Ollama real (generación de IA lenta).
- **Independent X** — `doc_id_prueba`` global y orden-dependente: `test_patch_document`` y `test_delete_document`` dependen de `test_subida_exitosa``. Aislados, fallan.
- **Repeatable X** — requieren infraestructura y un `.env`` que está en `.gitignore`` (líneas 105/130): en un clone fresco el import de `Settings`` falla.
- **Self-Validating X** — assertion stale + 500 no contemplado.
- **Timely X** — estructura de suite única sugiere tests posteriores a la implementación.

### ### 3. Anti-patterns del skill TDD

- **Horizontal slicing** — una suite de integración que testea todo junto, en vez de slices verticales (test → implementación → test).
- **Un solo seam** — solo se testea el seam HTTP (`TestClient``). Sin tests en `service/`` (`calcular_checksum``, `extraer_texto_real``, `generar_resumen_ia``) ni en `repository/``.
- **Sin mocks** — se llaman servicios externos reales (Ollama, MongoDB) en vez de mockearlos.

### ### 4. Las Tres Leyes de TDD no se cumplen

No hay ciclo red→green: la lógica de negocio (service y repository) no tuvo tests previos; los tests se agregaron como suite final de integración.

## **## Lo que sí está bien**

- Los tests usan la interfaz pública HTTP (el seam correcto), no internals.
- Cubren casos de negocio relevantes: duplicado (409), tamaño excesivo (400), 404.
- El README declara TDD como metodología del proyecto.
- ``test_error_tamano`` es autocontenido (valida antes de parsear el PDF).

## **## Recomendaciones para cumplir TDD**

1. Confirmar los seams a testear: HTTP (integración), service (unidad) y repository.
2. Mockear ``cliente_ia.generate`` (Ollama) en los tests HTTP.
3. Corregir la assertion stale y usar un PDF mínimo real para ``test_subida_exitosa``.
4. Fixtures independientes por test en ``conftest.py`` (eliminar ``global doc_id_prueba``).
5. Agregar ``env.example`` para que los tests corran en un clone fresco.
6. De acá en más: ciclo red→green por slice vertical (1 test → 1 implementación).